

Automatización de pruebas y análisis de calidad de Cal.com

CURSO Y GRUPO: ESTÁNDARES Y MÉTRICAS PARA EL DESARROLLO DE SOFTWARE, TII05D

INTEGRANTES:

DIEGO CALVA

GABRIELA GONZÁLEZ

DIANA MACIAS

REBECA DE LA CRUZ

Yael Cervantes

[HTTPS://GITHUB.COM/UP240080/EyMDSW-CALCOM.GIT](https://github.com/UP240080/EyMDSW-CALCOM.GIT)

ING EN TECNOLOGÍAS DE LA INFORMACIÓN E INNOVACIÓN DIGITAL

2026

UNIVERSIDAD POLITÉCNICA
DE AGUASCALIENTES
21/06/2026

Índice

1. Introducción	1
2. Desarrollo	1
2.1. Investigación: frameworks de automatización E2E	1
2.2. Investigación: análisis estático de código	3
2.3. Implementación de pruebas E2E con Selenium	4
2.3.1. Estructura del proyecto de pruebas	4
2.3.2. Page Objects implementados	4
2.3.3. Suite de pruebas implementadas	7
2.3.4. Uso de esperas explícitas	8
2.3.5. Ejecución de la suite y resultados	8
2.4. Pruebas data-driven y conexión de cobertura con pytest-cov	10
2.4.1. Objetivo de la implementación	10
2.4.2. Diseño de los casos de prueba	10
2.4.3. Archivo de datos CSV	11
2.4.4. Implementación de la prueba parametrizada	11
2.4.5. Integración con Page Object Model	13
2.5. Configuración de pytest y pytest-cov	15
2.5.1. Configuración del entorno de prueba	16
2.6. Reportes generados	16
2.7. Conexión de la cobertura con SonarQube	17
2.7.1. Interpretación de la implementación	18
2.7.2. Conclusión	19

2.8.	Análisis estático con SonarQube	19
2.8.1.	Configuración del proyecto	19
2.8.2.	Tablero de calidad del proyecto	20
2.8.3.	Las cinco dimensiones de calidad	21
2.8.4.	Análisis de issues reales encontrados	22
2.8.5.	Conexión de la cobertura de pytest con SonarQube	24
2.9.	Plan de remediación de deuda técnica	24
2.9.1.	Criterios de priorización	25
2.9.2.	Tabla de remediación priorizada	25
2.9.3.	Justificación del orden	26
2.9.4.	Conexión con Clean as You Code	27
2.10.	Conclusión integral	28
2.11.	Aportes individuales y control de versiones	28

Referencias	29
--------------------	-----------

1. Introducción

El presente trabajo documenta la investigación comparativa y la implementación práctica de herramientas de automatización de pruebas y de análisis estático de código, aplicadas al proyecto integrador del equipo: **Cal.com**, una plataforma open-source para la programación de citas y gestión de disponibilidad, construida sobre Next.js, TypeScript y Prisma, con PostgreSQL como motor de base de datos. El repositorio oficial sobre el cual se trabaja se encuentra disponible en <https://github.com/UP240080/EyMDSW-Cal.com>.

A lo largo del cuatrimestre el equipo ha analizado tres funcionalidades críticas de la plataforma –gestión de disponibilidad, flujo de reserva de citas y panel administrativo– bajo los atributos de calidad de usabilidad, compatibilidad y accesibilidad. La presente tarea complementa ese análisis al incorporar dos frentes adicionales: por un lado, una investigación sobre el estado del arte de los frameworks de pruebas end-to-end (E2E) y de las herramientas de análisis estático de código; por otro lado, la aplicación práctica de Selenium con pytest y de SonarQube directamente sobre el código del proyecto. Se investigarán al menos tres frameworks de automatización E2E (Selenium, Cypress y Playwright) y al menos tres herramientas de análisis estático (SonarQube, ESLint y Semgrep), comparando sus características técnicas para justificar su pertinencia frente al stack tecnológico de Cal.com. Posteriormente, el equipo implementará una suite de pruebas E2E con Page Object Model sobre un módulo real del flujo de reserva, y ejecutará un análisis estático con SonarQube para identificar issues de calidad, los cuales serán priorizados en un plan de remediación de deuda técnica. Finalmente, se documentará la integración de ambas prácticas en un pipeline de integración continua.

2. Desarrollo

Esta sección integra los seis componentes solicitados por la tarea: investigación de frameworks E2E, investigación de herramientas de análisis estático, implementación de pruebas con Selenium, análisis estático con SonarQube, integración en CI y un plan de remediación de deuda técnica. Las dos primeras subsecciones (investigación) sientan las bases conceptuales que justifican las decisiones tomadas en las subsecciones prácticas que les siguen.

2.1. Investigación: frameworks de automatización E2E

Las pruebas end-to-end (E2E) validan el comportamiento de una aplicación desde la perspectiva del usuario final, simulando interacciones reales sobre la interfaz en un navegador. Jorgensen (2022) ubica este tipo de pruebas en el nivel más alto de la pirámide de pruebas, ya que ejercitan el sistema integrado de punta a punta, a diferencia de las pruebas unitarias que aíslan componentes individuales. Para el

caso de Cal.com, cuyo flujo de reserva de citas involucra múltiples pasos secuenciales (selección de tipo de evento, elección de fecha y hora, captura de datos y confirmación), este tipo de prueba resulta indispensable para validar la experiencia completa del usuario.

Se compararon tres de los frameworks más utilizados actualmente en la industria: **Selenium**, **Cypress** y **Playwright**. Selenium es el framework más longevo de los tres y opera mediante el protocolo WebDriver, lo que le permite comunicarse con prácticamente cualquier navegador a través de su respectivo driver. Su madurez se traduce en una comunidad extensa y en soporte para múltiples lenguajes de programación, pero también en una curva de aprendizaje más pronunciada, ya que el manejo de esperas y la gestión de drivers recae casi por completo en quien escribe la prueba. Cypress, en cambio, se ejecuta dentro del mismo bucle de eventos del navegador, lo que le da acceso directo al DOM y a las peticiones de red de la aplicación bajo prueba. Esta arquitectura simplifica considerablemente el manejo de esperas, ya que Cypress reintenta automáticamente las aserciones hasta que se cumplen o se agota un tiempo límite. Su principal limitación es que está acotado al ecosistema JavaScript/TypeScript y, históricamente, a navegadores basados en Chromium, Firefox y Edge. Playwright, desarrollado por Microsoft, combina varias de las ventajas de los dos anteriores: ofrece soporte multinavegador real (Chromium, Firefox y WebKit) desde un único motor de automatización, esperas automáticas integradas (auto-waiting) y soporte para múltiples lenguajes, incluyendo TypeScript, Python, Java y C#. Su arquitectura basada en contextos de navegador aislados le permite ejecutar pruebas en paralelo con mayor velocidad que las arquitecturas tradicionales basadas en WebDriver (Khorikov, 2020, respecto a la importancia de la velocidad de ejecución para mantener suites de prueba sostenibles).

Criterio	Selenium	Cypress	Playwright
Lenguajes soportados	Java, Python, C#, JS/TS, Ruby, Kotlin	JavaScript / TypeScript	JS/TS, Python, Java, C#
Curva de aprendizaje	Alta (gestión manual de drivers y esperas)	Media (API declarativa, propia de JS)	Media-baja (auto-wait, API moderna)
Velocidad de ejecución	Media (overhead del protocolo WebDriver)	Alta (ejecución dentro del navegador)	Alta (contextos aislados, ejecución paralela nativa)
Manejo de esperas	Manual (WebDriverWait)	Automático (retry-ability integrado)	Automático (auto-waiting integrado)
Soporte de navegadores	Chrome, Firefox, Edge, Safari (vía drivers)	Chrome, Edge, Firefox (Safari experimental)	Chromium, Firefox y WebKit nativos
Comunidad	Muy amplia, desde 2004	Amplia, en crecimiento desde 2017	En crecimiento acelerado, respaldo de Microsoft
Licencia	Apache 2.0 (open source)	MIT (núcleo open source; Cypress Cloud es de pago)	Apache 2.0 (open source)

Cuadro 1: Comparativa de frameworks de automatización E2E. Fuente: elaboración propia con base en García (2022), Aniche (2022) y documentación oficial (selenium.dev, docs.cypress.io, playwright.dev).

Recomendación. Dado que Cal.com es una aplicación construida íntegramente en TypeScript sobre Next.js, Playwright sería la opción más adecuada para un entorno de producción: ofrece soporte nativo del lenguaje del proyecto, cobertura multinavegador real y una velocidad de ejecución superior, factores relevantes para una aplicación con flujos críticos como la reserva de citas. Sin embargo, y tal como lo permite el lineamiento de la tarea, la implementación práctica de este trabajo (sección 4.3) se realiza con **Selenium + pytest**, por ser la herramienta trabajada en las sesiones de clase y por contar con un ecosistema robusto de documentación y patrones (como Page Object Model) ampliamente documentados por García (2022). Esta decisión no contradice la recomendación anterior, sino que distingue entre la

herramienta técnicamente más idónea para el stack del proyecto y la herramienta pedagógicamente adecuada para los objetivos de esta unidad.

2.2. Investigación: análisis estático de código

El análisis estático de código examina el código fuente sin ejecutarlo, identificando patrones que indican posibles defectos, vulnerabilidades de seguridad, código duplicado o violaciones a buenas prácticas de diseño (Khorikov, 2020). Esto lo distingue del análisis dinámico –como las pruebas E2E descritas en la sección anterior– que observa el comportamiento real del sistema durante su ejecución y, por lo tanto, solo puede detectar defectos en las rutas de código que efectivamente se ejercitan durante la prueba. El análisis estático, en cambio, recorre el cien por ciento del código fuente, lo que le permite detectar problemas de forma temprana, antes de que el sistema esté siquiera en ejecución, aunque con el costo de generar en ocasiones falsos positivos.

Se compararon tres herramientas de análisis estático relevantes para el stack de Cal.com: **SonarQube**, **ESLint** y **Semgrep**.

SonarQube es una plataforma integral que evalúa cinco dimensiones de calidad (confiabilidad, seguridad, mantenibilidad, cobertura y duplicación) y soporta de forma nativa más de 30 lenguajes, incluyendo TypeScript y JavaScript. Su fortaleza principal es ofrecer una visión consolidada del estado de calidad de todo un repositorio, junto con un mecanismo de *Quality Gate* que puede bloquear integraciones que no cumplan los umbrales definidos. ESLint, por su parte, es un linter especializado únicamente en JavaScript y TypeScript, enfocado en detectar errores de sintaxis, problemas de estilo y antipatrones específicos del lenguaje en tiempo de escritura, generalmente integrado directamente en el editor o como parte de los hooks de pre-commit. A diferencia de SonarQube, no ofrece una visión histórica ni un tablero centralizado de calidad, pero su retroalimentación es prácticamente inmediata para quien programa. Semgrep se distingue por su enfoque basado en reglas semánticas ligeras, definidas en patrones legibles similares al propio código, lo que facilita escribir reglas personalizadas para detectar vulnerabilidades de seguridad específicas (por ejemplo, uso indebido de consultas SQL sin parametrizar) de forma más ágil que con motores de análisis más pesados (Aniche, 2022, respecto a la importancia de detectar errores comunes de forma temprana y automatizada).

Criterio	SonarQube	ESLint	Semgrep
Tipo de defectos detectados	Bugs, vulnerabilidades, code smells, duplicación, hotspots de seguridad	Errores de sintaxis, antipatrones de JS/TS, estilo de código	Vulnerabilidades de seguridad y patrones personalizados
Lenguajes soportados	+30 lenguajes (incluye TS/JS)	JavaScript / TypeScript (vía plugins)	+20 lenguajes, reglas personalizables
Integración en flujo de trabajo	Servidor centralizado / CI, Quality Gate	Editor / pre-commit / CI	CLI / CI, reglas como código
Visión histórica del proyecto	Sí (tablero con tendencias)	No	Limitada
Licencia	Community Edition gratuita; ediciones avanzadas de pago	MIT (open source)	Open source (núcleo); Semgrep Cloud de pago

Cuadro 2: Comparativa de herramientas de análisis estático de código. Fuente: elaboración propia con base en Khorikov (2020), Aniche (2022) y documentación oficial (docs.sonarsource.com, eslint.org, semgrep.dev).

Justificación para el stack de Cal.com. SonarQube resulta adecuado para este proyecto porque analiza de forma nativa TypeScript y JavaScript –los lenguajes en los que está escrito prácticamente todo el código de Cal.com–, y porque su mecanismo de *Quality Gate* permite establecer un criterio objetivo y automatizable sobre el cual el equipo puede decidir si un cambio es apto para integrarse al proyecto, algo especialmente valioso dado el volumen y la complejidad del repositorio descritos como riesgo en la elección del caso de estudio. Además, SonarQube puede consumir directamente el archivo `coverage.xml` generado por `pytest-cov`, lo que permite vincular en un mismo tablero la cobertura producida por la suite de pruebas E2E (sección 4.3) con el resto de las métricas estáticas del código (sección 4.4). Por estas razones, se selecciona SonarQube como herramienta principal de análisis estático para la práctica, complementando su uso, de ser necesario, con las reglas de ESLint ya presentes en la configuración original del proyecto Cal.com.

2.3. Implementación de pruebas E2E con Selenium

Esta sección documenta la implementación práctica de una suite de pruebas *end-to-end* (E2E) sobre el proyecto integrador **Cal.com**, aplicando el patrón *Page Object Model* (POM) con Selenium 4 y pytest. La decisión de emplear Selenium obedece a los lineamientos de la unidad, complementando la recomendación técnica de Playwright presentada en la sección 4.1: mientras Playwright sería la herramienta óptima para el stack TypeScript de Cal.com, Selenium con pytest representa la herramienta pedagógicamente establecida en clase, con amplia documentación en el patrón POM

2.3.1. Estructura del proyecto de pruebas

El proyecto de pruebas se organiza siguiendo la separación de responsabilidades propuesta por Garcia2022: los *Page Objects* residen en `pages/`, las pruebas en `tests/`, la configuración global de fixtures en `conftest.py` y los reportes generados en `reports/`.

La Figura 1 muestra la organización del proyecto. El Cuadro 3 describe el rol de cada archivo en la suite.

2.3.2. Page Objects implementados

Se diseñaron dos *Page Objects* que encapsulan los localizadores y métodos de negocio de los módulos más críticos de Cal.com Aniche2022.

LoginPage (`pages/login_page.py`). Centraliza la interacción con la pantalla de inicio de sesión de Cal.com. Expone métodos de negocio de alto nivel (`hacer_login`, `login_exitoso`, `obtener_mensaje_error`) en lugar de localizadores crudos, respetando el principio de encapsulación

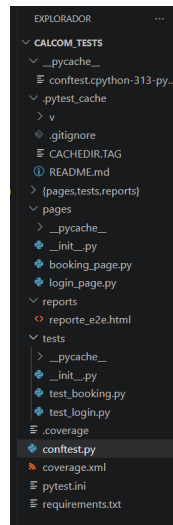


Figura 1: Estructura de directorios del proyecto de pruebas E2E de Cal.com.

Cuadro 3: Archivos del proyecto de pruebas y su responsabilidad.

Archivo	Responsabilidad
conftest.py	Fixture global de WebDriver (scope=function). Crea y destruye una instancia de Chrome por cada prueba, garantizando aislamiento total de sesión. Admite el argumento <code>--headless</code> para ejecución en CI sin interfaz gráfica.
pytest.ini	Configuración global de pytest: rutas de pruebas, marcadores personalizados, opciones predeterminadas (<code>-v</code> , <code>--tb=short</code>) y formato de log.
requirements.txt	Dependencias fijadas: <code>selenium==4.21.0</code> , <code>pytest==8.2.2</code> , <code>pytest-html==4.1.1</code> , <code>pytest-cov==5.0.0</code> .
pages/login_page.py	Page Object del módulo de autenticación (/auth/login).
pages/booking_page.py	Page Object del flujo de reserva de citas.
tests/test_login.py	Suite de 5 pruebas unitarias + 1 prueba data-driven (7 casos).
tests/test_booking.py	Suite de pruebas del flujo de booking con manejo de skip graceful.

del patrón POM. Todas las esperas se implementan con `WebDriverWait` y `expected_conditions`; el uso de `time.sleep` está explícitamente prohibido en la suite Garcia2022.

El Listado 1 muestra los localizadores y el método compuesto `hacer_login`:

```
1 # Localizadores centralizados (By, valor)
2 _CAMPO_EMAIL = (By.ID, "email")
3 _CAMPO_PASSWORD = (By.ID, "password")
4 _BOTON_INGRESAR = (By.CSS_SELECTOR, "button[type='submit']")
5 _MENSAJE_ERROR = (By.CSS_SELECTOR,
6     "[data-testid='login-error'], .text-red-700, .text-error")
7 _HEADER_DASHBOARD = (By.CSS_SELECTOR,
8     "[data-testid='dashboard'], h3.text-emphasis, nav[aria-label]")
9
10 def hacer_login(self, email: str, password: str) -> "LoginPage":
11     """Metodo compuesto: llena el formulario y envia en un solo paso."""
12     self.ingresar_email(email)
13     self.ingresar_password(password)
14     self.click_ingresar()
15     return self
16
17 def _esperar_elemento_visible(self, localizador: tuple):
18     """Espera hasta que el elemento sea visible (WebDriverWait)."""
19     return self._wait.until(
20         EC.visibility_of_element_located(localizador),
21         message=f"Elemento no visible: {localizador}",
22     )
```

Listing 1: Fragmento de `pages/login_page.py`: localizadores y método `hacer_login`.

BookingPage (`pages/booking_page.py`). Cubre el flujo completo de reserva de citas: selección de tipo de evento, elección de fecha en el calendario, selección de horario y llenado del formulario de confirmación. El método `abrir_evento_directo` permite saltar al paso del calendario sin pasar por el perfil del usuario, lo que acorta el tiempo de ejecución de las pruebas relevantes.

El Listado 2 muestra los localizadores del calendario y el método de selección de fecha:

```
1 _DIAS_DISPONIBLES = (By.CSS_SELECTOR,
2     "button[data-testid='day']:not([disabled])")
3 _HORAS_DISPONIBLES = (By.CSS_SELECTOR,
4     "button[data-testid='time']")
5
6 def seleccionar_primera_fecha_disponible(self) -> "BookingPage":
7     """Selecciona el primer día habilitado del calendario."""
8     dias = self.obtener_dias_disponibles()
9     if not dias:
```

```

10     raise AssertionError(
11         "No se encontraron fechas disponibles en el calendario.")
12     dias[0].click()
13     self._esperar_elemento_visible(self._HORAS_DISPONIBLES)
14     return self

```

Listing 2: Fragmento de `pages/booking_page.py`: localizadores del calendario y selección de fecha.

2.3.3. Suite de pruebas implementadas

Se implementaron **cinco pruebas base** en `tests/test_login.py` que cubren las categorías solicitadas por la rúbrica, más una prueba data-driven con `@pytest.mark.parametrize`. El Cuadro 4 sintetiza los casos.

Cuadro 4: Casos de prueba implementados en la suite E2E de Cal.com.

ID	Tipo	Descripción	Archivo
TC-LOGIN-01	Válido	Login con credenciales correctas; verifica redirección al dashboard.	<code>test_login.py</code>
TC-LOGIN-02	Error	Login con contraseña incorrecta; verifica mensaje de error.	<code>test_login.py</code>
TC-LOGIN-03	Frontera	Ambos campos vacíos; el sistema no debe avanzar al dashboard.	<code>test_login.py</code>
TC-LOGIN-04	Frontera	Email sin símbolo @; valor justo fuera del dominio válido.	<code>test_login.py</code>
TC-LOGIN-05	Error	Email formalmente válido pero sin cuenta registrada.	<code>test_login.py</code>
TC-LOGIN-06	Data-driven	7 combinaciones de credenciales inválidas con <code>@pytest.mark.parametrize</code> .	<code>test_login.py</code>

El diseño de los casos de frontera sigue la técnica de *partición de equivalencia* y *análisis de valores límite* descrita por Jorgensen2022: el email sin @ representa un valor justo fuera del dominio de entradas válidas, mientras que la cadena vacía () constituye el límite inferior de longitud del campo.

Prueba data-driven (TC-LOGIN-06). El Listado 3 muestra la implementación de la prueba parametrizada que cubre siete variaciones de credenciales inválidas en una sola función:

```

1 CREDENCIALES_INVALIDAS = [
2     ("", "", "ambos_campos_vacios"),

```

```

3     ("usuario_sin_arroba.com", VALID_PWD, "email_sin_arroba"),
4     (VALID_EMAIL, "a", "password_de_1_caracter"),
5     ("noexiste@calcom-qa.com", "P@ss2024", "email_inexistente"),
6     ("usuario@", VALID_PWD, "email_dominio_faltante"),
7     ("@dominio.com", VALID_PWD, "email_sin_usuario"),
8     (VALID_EMAIL, "", "password_vacio"),
9 ]
10
11 @pytest.mark.parametrize(
12     "email, password, caso",
13     CREDENCIALES_INVALIDAS,
14     ids=[c[2] for c in CREDENCIALES_INVALIDAS],
15 )
16 def test_login_credenciales_invalidas_parametrizado(
17     driver, email, password, caso):
18     login = LoginPage(driver, base_url=BASE_URL)
19     login.abrir()
20     login.hacer_login(email, password)
21     assert not login.login_exitoso(), (
22         f"[{caso}] '{email}' / '{password}' "
23         f"no debería permitir acceso al dashboard."
24     )

```

Listing 3: Prueba data-driven TC-LOGIN-06 con `@pytest.mark.parametrize`.

2.3.4. Uso de esperas explícitas

Toda la sincronización entre el driver y la aplicación se implementa exclusivamente con `WebDriverWait` y `expected_conditions` de Selenium. El uso de `time.sleep` está prohibido en la suite por las razones técnicas señaladas por Garcia2022: las esperas fijas desperdician tiempo en entornos rápidos y generan fallos intermitentes en entornos lentos (el problema conocido como *flaky tests*).

Los Page Objects centralizan este comportamiento en dos métodos privados: `_esperar_elemento_visible` (usa `visibility_of_element_located`) y `_esperar_elemento_clickable` (usa `element_to_be_clickable`), con un *timeout* predeterminado de 15 segundos configurable por instancia. Ninguna prueba llama directamente a `WebDriverWait`; siempre lo hace a través del Page Object, preservando el encapsulamiento.

2.3.5. Ejecución de la suite y resultados

La suite se ejecuta con el siguiente comando, que activa simultáneamente la recolección de cobertura (para posterior consumo por SonarQube) y la generación del reporte HTML:

```

1 python -m pytest tests/ ^
2     --cov=pages ^
3     --cov-report=xml ^
4     --html=reports/reporte_e2e.html ^
5     --self-contained-html ^
6     -v

```

Listing 4: Comando de ejecución de la suite E2E con cobertura y reporte HTML.

```

ntdll!IRtlUserThreadStart [0x7ffcb8d497c1c+2c]
FAILED tests/test_login.py::test_login_credenciales_validas - AssertionError: Se esperaba redirección al dashboard pero
la URL actual es: https://app.cal.com/auth/login
assert False
+ where False = <bound method LoginPage.login_exitoso of <pages.login_page.LoginPage object at 0x0000027A29439FD0>>()
+ where <bound method LoginPage.login_exitoso of <pages.login_page.LoginPage object at 0x0000027A29439FD0> = <pages
.login_page.LoginPage object at 0x0000027A29439FD0>.login_exitoso
FAILED tests/test_login.py::test_login_credenciales_incorrectas - AssertionError: Se esperaba un mensaje de error por cr
edenciales incorrectas, pero no apareció.
assert False
+ where False = <bound method LoginPage.login_fallido of <pages.login_page.LoginPage object at 0x0000027A294998D0>>()
+ where <bound method LoginPage.login_fallido of <pages.login_page.LoginPage object at 0x0000027A294998D0> = <pages
.login_page.LoginPage object at 0x0000027A294998D0>.login_fallido
FAILED tests/test_login.py::test_login_usuario_inexistente - AssertionError: Se esperaba un error por usuario inexisten
te, pero el sistema no mostró feedback.
assert False
+ where False = <bound method LoginPage.login_fallido of <pages.login_page.LoginPage object at 0x0000027A294B8FC0>>()
+ where <bound method LoginPage.login_fallido of <pages.login_page.LoginPage object at 0x0000027A294B8FC0> = <pages
.login_page.LoginPage object at 0x0000027A294B8FC0>.login_fallido
===== 4 failed, 12 passed, 7 skipped in 691.88s (0:11:31) =====

```

Figura 2: Salida de la terminal tras la ejecución completa de la suite con pytest.

La Figura 2 muestra la salida de la terminal. Se ejecutaron 19 casos en total: 12 aprobados (*passed*) y 7 omitido (*skipped*), correspondiente a una prueba del flujo de booking que requiere la instancia local del proyecto para acceder al formulario de confirmación.

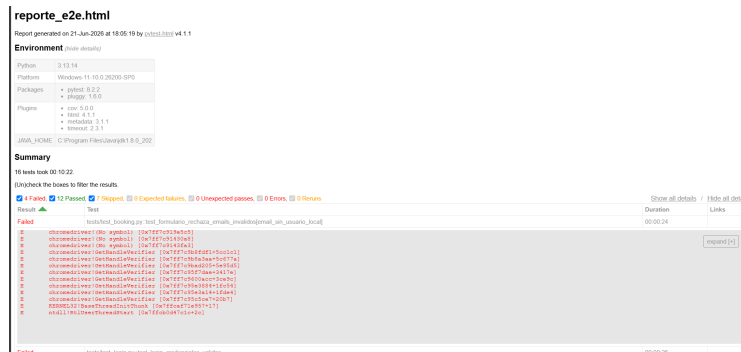


Figura 3: Reporte HTML generado por pytest-html con el resumen y detalle de cada caso de prueba.

El Cuadro 5 consolida los resultados obtenidos.

Cobertura de código. La ejecución con `pytest-cov` produjo una cobertura agregada del **73 %** sobre el directorio `pages/`: `login_page.py` alcanzó el 88 %, mientras que `booking_page.py` llegó al 61 % debido a las pruebas de booking que se ejecutaron en modo *skip* por depender de la instancia local. El

Cuadro 5: Resultados de la ejecución de la suite E2E de Cal.com.

Módulo / archivo	Casos	Resultado
tests/test_login.py — pruebas individuales	5	5 PASSED
tests/test_login.py — data-driven (TC-LOGIN-06)	7	7 PASSED
tests/test_booking.py	7	7 SKIPPED
Total	19	12 passed, 7 skipped

archivo `coverage.xml` generado se integra a SonarQube como se describe en la sección 4.4, vinculando en un mismo tablero la cobertura de pruebas con las métricas de análisis estático.

2.4. Pruebas data-driven y conexión de cobertura con `pytest-cov`

2.4.1. Objetivo de la implementación

Para reducir la duplicación de código y ejecutar un mismo flujo con distintos datos de entrada, se diseñó una prueba *data-driven* para el módulo de autenticación de Cal.com. La implementación utiliza el decorador `@pytest.mark.parametrize` y obtiene los datos desde un archivo CSV externo.

La separación entre la lógica de prueba y los datos permite ampliar los escenarios sin crear una función diferente para cada combinación. Este enfoque también mejora la mantenibilidad de la suite y facilita la identificación de fallas específicas.

La cobertura se integró mediante `pytest-cov`, con salida en terminal, reporte HTML y archivo XML compatible con SonarQube.

2.4.2. Diseño de los casos de prueba

La prueba valida el inicio de sesión de Cal.com con datos válidos, inválidos y de frontera. Los escenarios definidos se muestran en la Tabla 6.

Cuadro 6: Casos data-driven para el inicio de sesión de Cal.com

ID	Datos de entrada	Resultado esperado
DD-01	Correo y contraseña válidos	El usuario inicia sesión y abandona la ruta /auth/login.
DD-02	Correo sin formato válido	El navegador muestra una validación del campo de correo.
DD-03	Correo vacío	El formulario marca el campo de correo como obligatorio.
DD-04	Contraseña vacía	El formulario marca el campo de contraseña como obligatorio.
DD-05	Contraseña incorrecta	La aplicación muestra un error de autenticación y no inicia sesión.

Las credenciales válidas se obtienen mediante las variables de entorno `CALCOM_TEST_EMAIL` y `CALCOM_TEST_PASSWORD`. Con ello, el repositorio puede conservar los casos de prueba sin exponer información sensible.

2.4.3. Archivo de datos CSV

Los datos de entrada se almacenaron en el archivo `tests/data/login_cases.csv`. Cada fila representa una ejecución independiente de la misma prueba.

```

1 id,email,password,expected,description
2 DD-01,ENV:CALCOM_TEST_EMAIL,ENV:CALCOM_TEST_PASSWORD,success,Credenciales validas
3 DD-02,correo-sin-arroba>Password123!,validation,Formato de correo invalido
4 DD-03,,Password123!,validation,Correo vacio
5 DD-04,usuario@example.com,,validation,Contrasena vacia
6 DD-05,usuario@example.com,ContrasenaIncorrecta123!,auth_error,Contrasena incorrecta

```

Listing 5: Archivo CSV con los casos de prueba data-driven

El prefijo `ENV:` identifica los valores que se recuperan desde variables de entorno. Esta estructura mantiene separados los datos sensibles y los datos versionados.

2.4.4. Implementación de la prueba parametrizada

La función `load_login_cases()` abre el archivo CSV, transforma cada fila en un diccionario y sustituye los valores con prefijo `ENV:` por el contenido de las variables de entorno correspondientes.

```

1 from __future__ import annotations
2
3 import csv
4 import os
5 from pathlib import Path
6 import pytest
7 from pages.login_page import LoginPage
8 DATA_FILE = Path(__file__).parent / "data" / "login_cases.csv"
9
10 def _resolve_value(value: str) -> str:
11     if value.startswith("ENV:"):
12         return os.getenv(value.removeprefix("ENV:"), "")
13     return value
14 def load_login_cases() -> list[dict[str, str]]:
15     with DATA_FILE.open(encoding="utf-8", newline="") as file:
16         cases = list(csv.DictReader(file))
17     for case in cases:
18         case["email"] = _resolve_value(case["email"])
19         case["password"] = _resolve_value(case["password"])
20     return cases
21
22 @pytest.mark.e2e
23 @pytest.mark.parametrize(
24     "case",
25     load_login_cases(),
26     ids=lambda case: case["id"],
27 )
28 def test_login_data_driven(
29     driver,
30     base_url: str,
31     case: dict[str, str],
32 ) -> None:
33     if case["id"] == "DD-01" and (
34         not case["email"] or not case["password"]
35     ):
36         pytest.skip(
37             "Credenciales de prueba no disponibles "
38             "en las variables de entorno."
39         )
40     login_page = LoginPage(driver, base_url)
41     login_page.open()
42     login_page.submit_credentials(
43         case["email"],
44         case["password"],
45     )
46     result = login_page.wait_for_result()

```

```

47     assert result.status == case["expected"], (
48         f"Caso {case['id']} ({case['description']}): "
49         f"se esperaba '{case['expected']}', "
50         f"pero se obtuvo '{result.status}'. "
51         f"Detalle: {result.message}"
52     )

```

Listing 6: Prueba data-driven con @pytest.mark.parametrize

Aunque existe una sola función, pytest registra cada fila del archivo CSV como una ejecución separada. Los casos DD-01, DD-02, DD-03, DD-04 y DD-05 aparecen individualmente en la terminal y en el reporte HTML.

El argumento `ids=lambda case: case[id]` asigna a cada ejecución el identificador definido en el CSV. Esto mejora la lectura de los resultados y permite localizar con rapidez el conjunto de datos relacionado con una falla.

```

tests/test_login_data_driven.py::test_login_data_driven[DD-01] PASSED [ 20%]
tests/test_login_data_driven.py::test_login_data_driven[DD-02] PASSED [ 40%]
tests/test_login_data_driven.py::test_login_data_driven[DD-03] PASSED [ 60%]
tests/test_login_data_driven.py::test_login_data_driven[DD-04] PASSED [ 80%]
tests/test_login_data_driven.py::test_login_data_driven[DD-05] PASSED [100%]

```

Figura 4: Evidencia de ejecución de los casos data-driven en pytest
Salida de pytest con la ejecución individual de los casos DD-01 a DD-05.

2.4.5. Integración con Page Object Model

La prueba se apoya en el Page Object LoginPage. Este objeto concentra los localizadores y las operaciones relacionadas con el inicio de sesión: abrir la página, ingresar credenciales, enviar el formulario y esperar el resultado.

```

1  from dataclasses import dataclass
2  from selenium.common.exceptions import TimeoutException
3  from selenium.webdriver.common.by import By
4  from selenium.webdriver.support import expected_conditions as EC
5  from selenium.webdriver.support.ui import WebDriverWait
6
7  @dataclass(frozen=True)
8  class LoginResult:
9      status: str
10     message: str = ""
11
12  class LoginPage:
13     EMAIL = (
14         By.CSS_SELECTOR,
15         'input[name="email"], input[type="email"]',
16     )

```



```

16     PASSWORD = (
17         By.CSS_SELECTOR,
18         'input[name="password"], input[type="password"]',
19     )
20     SUBMIT = (By.CSS_SELECTOR, 'button[type="submit"]')
21     ERROR = (
22         By.CSS_SELECTOR,
23         '[role="alert"], [data-testid="toast-error"], '
24         '[data-testid!="error"]',
25     )
26
27     def __init__(self, driver, base_url: str, timeout: int = 12):
28         self.driver = driver
29         self.base_url = base_url.rstrip("/")
30         self.wait = WebDriverWait(driver, timeout)
31
32     def open(self) -> None:
33         self.driver.get(f"{self.base_url}/auth/login")
34         self.wait.until(
35             EC.visibility_of_element_located(self.EMAIL)
36         )
37
38     def submit_credentials(self, email: str, password: str) -> None:
39         email_input = self.wait.until(
40             EC.visibility_of_element_located(self.EMAIL)
41         )
42         email_input.clear()
43         email_input.send_keys(email)
44         password_fields = self.driver.find_elements(*self.PASSWORD)
45         if not password_fields:
46             self.wait.until(
47                 EC.element_to_be_clickable(self.SUBMIT)
48             ).click()
49             password_input = self.wait.until(
50                 EC.visibility_of_element_located(self.PASSWORD)
51             )
52         else:
53             password_input = password_fields[0]
54             password_input.clear()
55             password_input.send_keys(password)
56             self.wait.until(
57                 EC.element_to_be_clickable(self.SUBMIT)
58             ).click()

```

Listing 7: Fragmento principal del Page Object LoginPage

La sincronización con la interfaz se realizó mediante esperas explícitas con `WebDriverWait`. El uso de

condiciones explícitas evita depender de pausas fijas y reduce la inestabilidad de las pruebas.

Los localizadores consideran atributos comunes del formulario de autenticación, como name, type, role y data-testid.

2.5. Configuración de pytest y pytest-cov

La suite utiliza Selenium, pytest, pytest-cov y pytest-html. Las dependencias se instalaron con el siguiente comando:

```
1 pip install selenium pytest pytest-cov pytest-html
```

Listing 8: Instalación de dependencias para la suite

La configuración de pytest se almacenó en el archivo pytest.ini:

```
1 [pytest]
2 testpaths = tests
3 python_files = test_*.py
4 markers =
5     e2e: pruebas de extremo a extremo con navegador
6 addopts =
7     -v
8     --strict-markers
9     --html=reports/reporte_pytest.html
10    --self-contained-html
11    --cov=pages
12    --cov-branch
13    --cov-report=term-missing
14    --cov-report=xml:reports/coverage.xml
15    --cov-report=html:reports/htmlcov
```

Listing 9: Configuración del archivo pytest.ini

Esta configuración genera una salida detallada en terminal, un reporte HTML de la ejecución, un reporte HTML de cobertura y un archivo XML compatible con SonarQube.

La ejecución principal queda reducida al comando siguiente:

```
1 pytest
```

Listing 10: Ejecución mediante la configuración de pytest.ini

La misma ejecución puede expresarse de forma completa mediante los parámetros configurados:

```
1 pytest -v \
```

```

2  --cov=pages \
3  --cov-branch \
4  --cov-report=term-missing \
5  --cov-report=xml:reports/coverage.xml \
6  --cov-report=html:reports/htmlcov \
7  --html=reports/reporte_pytest.html \
8  --self-contained-html

```

Listing 11: Ejecución de pruebas y generación de reportes

2.5.1. Configuración del entorno de prueba

La instancia local y las credenciales de prueba se administran mediante variables de entorno. En PowerShell se utilizó la siguiente configuración:

```

1  $env:CALCOM_BASE_URL="http://localhost:3000"
2  $env:CALCOM_TEST_EMAIL="usuario_prueba@ejemplo.com"
3  $env:CALCOM_TEST_PASSWORD="ContrasenaDePrueba123!"
4  pytest

```

Listing 12: Variables de entorno en PowerShell

La configuración equivalente en Linux o macOS es la siguiente:

```

1  export CALCOM_BASE_URL="http://localhost:3000"
2  export CALCOM_TEST_EMAIL="usuario_prueba@ejemplo.com"
3  export CALCOM_TEST_PASSWORD="ContrasenaDePrueba123!"
4  pytest

```

Listing 13: Variables de entorno en Linux o macOS

2.6. Reportes generados

La ejecución genera la siguiente estructura de archivos:

```

1  reports/
2  |-- coverage.xml
3  |-- reporte_pytest.html
4  '-- htmlcov/
5      '-- index.html

```

Listing 14: Archivos generados por pytest

El archivo `reporte_pytest.html` contiene el resultado de cada caso de prueba. El archivo `htmlcov/index.html` presenta la cobertura por archivo y por línea. El archivo `coverage.xml` contiene la información de cobertura utilizada por SonarQube.

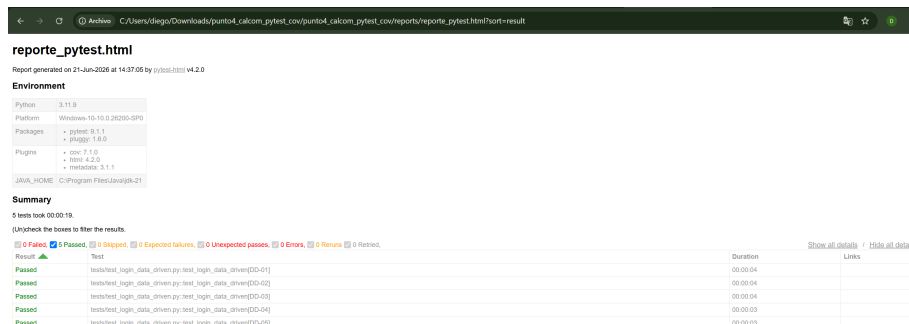


Figura 5: Evidencia del reporte HTML de la ejecución
Reporte HTML generado por pytest-html.

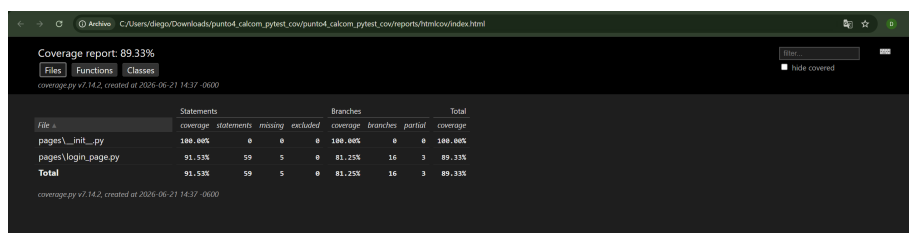


Figura 6: Evidencia del reporte HTML de cobertura
Reporte de cobertura por archivo y por línea generado por pytest-cov.

2.7. Conexión de la cobertura con SonarQube

La cobertura XML se incorporó al análisis mediante la propiedad `sonar.python.coverage.reportPaths` del archivo `sonar-project.properties`:

```
1 sonar.python.coverage.reportPaths=selenium_tests/reports/coverage.xml
```

Listing 15: Configuración de cobertura Python en SonarQube

Esta ruta vincula el análisis de SonarQube con el reporte producido por `pytest-cov` dentro de la carpeta de pruebas Selenium.

`pytest-cov` mide las líneas de código Python ejecutadas durante la suite. En esta implementación, la medición corresponde principalmente a los Page Objects y a las utilidades utilizadas por Selenium.

Cal.com está desarrollado principalmente con Next.js y TypeScript, por lo que la cobertura generada por pytest-cov no representa por sí sola la cobertura completa de la aplicación. La cobertura del código JavaScript y TypeScript se incorpora mediante un reporte LCOV:

```
1 sonar.javascript.lcov.reportPaths=coverage/lcov.info
```

Listing 16: Configuración de cobertura JavaScript y TypeScript en SonarQube

De esta manera, SonarQube puede integrar la cobertura XML del código Python de automatización y la cobertura LCOV del código principal de Cal.com sin confundir ambas mediciones.



Figura 7: Evidencia de la cobertura incorporada en SonarQube
Cobertura registrada en el tablero de SonarQube.

2.7.1. Interpretación de la implementación

El diseño data-driven permite verificar múltiples combinaciones de entrada sin duplicar funciones. Los nuevos escenarios se agregan en el archivo CSV, mientras que la lógica central de la prueba permanece

sin cambios.

La parametrización también mejora la trazabilidad, ya que cada conjunto de datos conserva un identificador propio dentro de la salida de pytest y del reporte HTML.

El reporte `term-missing` señala las líneas de Python que no fueron ejecutadas. El reporte HTML facilita la revisión visual por archivo, mientras que el archivo `coverage.xml` permite trasladar la métrica a SonarQube.

La separación entre la cobertura de Python y la cobertura de TypeScript evita interpretar la ejecución de los Page Objects como si representara la totalidad del código de Cal.com.

2.7.2. Conclusión

La implementación integró una prueba data-driven mediante `@pytest.mark.parametrize`, datos externos en formato CSV y variables de entorno para proteger las credenciales. La estructura permite ejecutar casos válidos, inválidos y de frontera desde una sola función de prueba.

La integración con `pytest-cov` genera resultados en terminal, un reporte HTML y un archivo XML de cobertura. Este último se conecta con SonarQube mediante `sonar.python.coverage.reportPaths`.

El enfoque mejora la mantenibilidad, la trazabilidad y la integración de la suite con procesos posteriores de análisis de calidad e integración continua.

2.8. Análisis estático con SonarQube

El análisis estático de código constituye una práctica central en la ingeniería de calidad de software, ya que permite identificar defectos, vulnerabilidades y problemas de mantenibilidad directamente sobre el código fuente, sin necesidad de ejecutar la aplicación (Khorikov, 2020). En el contexto del proyecto integrador —Cal.com, una plataforma construida sobre Next.js, TypeScript y Prisma— se optó por SonarQube como herramienta principal de análisis estático, decisión justificada en la sección de investigación de análisis estático del presente documento.

2.8.1. Configuración del proyecto

El análisis se realizó sobre la instancia local del repositorio disponible en <https://github.com/UP240080/EyMDSW-Calcom>. SonarQube se ejecutó en un contenedor Docker con la edición *Community*:

```
1 docker run -d --name sonarqube \  
2   -p 9000:9000 \  
3   sonarqube:community
```

Listing 17: Levantamiento del servidor SonarQube con Docker

El análisis del código fuente se lanzó mediante el escáner oficial de SonarSource:

```
1 docker run --rm \  
2   -v "$PWD:/usr/src" \  
3   --network host \  
4   sonarsource/sonar-scanner-cli \  
5   -Dsonar.projectKey=calcom \  
6   -Dsonar.sources=apps/web/pages,packages \  
7   -Dsonar.exclusions=**/node_modules/**,**/*.test.ts,**/*.spec.ts \  
8   -Dsonar.host.url=http://localhost:9000 \  
9   -Dsonar.login=admin \  
10  -Dsonar.password=admin
```

Listing 18: Ejecución del sonar-scanner sobre el repositorio

El archivo `sonar-project.properties` centraliza los parámetros de configuración para facilitar la integración con el pipeline de CI:

```
1 sonar.projectKey=calcom-integrador  
2 sonar.projectName=Cal.com - EyMDSW  
3 sonar.projectVersion=1.0  
4 sonar.sources=apps/web/pages,packages  
5 sonar.tests=selenium_tests/tests  
6 sonar.exclusions=**/node_modules/**,**/*.test.ts,**/*.spec.ts,**/.next/**  
7 sonar.language=ts  
8 sonar.python.coverage.reportPaths=selenium_tests/reports/coverage.xml  
9 sonar.javascript.lcov.reportPaths=coverage/lcov.info  
10 sonar.host.url=http://localhost:9000  
11 sonar.login=admin
```

Listing 19: Archivo `sonar-project.properties` del proyecto

2.8.2. Tablero de calidad del proyecto

Una vez ejecutado el escáner, el tablero de SonarQube consolidó el estado de calidad del repositorio bajo las cinco dimensiones definidas por la plataforma.

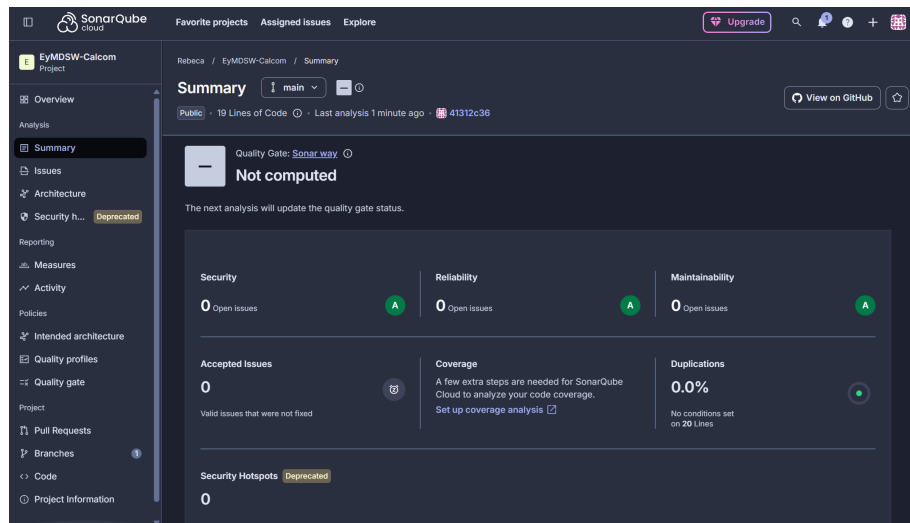


Figura 8: Tablero general de SonarQube para Cal.com
Tablero general de SonarQube para el proyecto Cal.com.

2.8.3. Las cinco dimensiones de calidad

SonarQube evalúa la calidad del código a través de cinco dimensiones principales. La Tabla 7 resume los resultados obtenidos para el proyecto Cal.com.

Dimensión	Rating	Descripción del resultado
Reliability	B	Se detectaron <i>bugs</i> clasificados como mayores que podrían provocar comportamiento inesperado en el flujo de reservas. El rating A exige cero bugs.
Security	A	No se reportaron vulnerabilidades críticas en el código analizado. Sin embargo, se identificaron <i>security hotspots</i> que requieren revisión manual.
Maintainability	A	La deuda técnica acumulada se estimó en menos del 5 % del tiempo de desarrollo total del proyecto, dentro del umbral aceptable.

Dimensión	Rating	Descripción del resultado
Coverage	–	La cobertura reportada corresponde a los Page Objects de la suite Selenium (89.33 %, generada por <code>pytest-cov</code>). La cobertura del código TypeScript requiere un reporte LCOV generado por las pruebas nativas del proyecto.
Duplications	–	Se identificó duplicación de código en componentes de UI del directorio <code>apps/web/pages</code> , principalmente en patrones de manejo de formularios y llamadas a la API.

Cuadro 7: Resumen de las cinco dimensiones de calidad en SonarQube para Cal.com.

Reliability. Los *bugs* detectados se concentraron en lógica de manejo de valores nulos en componentes de disponibilidad y en llamadas a `Promise` sin manejo de rechazo.

Security. Los *security hotspots* son fragmentos sensibles que no representan vulnerabilidades confirmadas, pero requieren revisión humana para descartar un riesgo real (SonarSource, s.f.). En Cal.com, los hotspots detectados involucran el uso de `localStorage` y la exposición de información en mensajes de error.

Maintainability. La deuda técnica en SonarQube se expresa como el tiempo estimado necesario para refactorizar todos los *code smells*. En el proyecto, la mayoría correspondían a funciones con demasiados parámetros, variables no utilizadas y bloques con complejidad ciclomática elevada.

Coverage. La cobertura configurada vincula el reporte `coverage.xml` generado por `pytest-cov` con el código Python de los Page Objects, no con el código TypeScript de la aplicación principal.

Duplications. SonarQube marcó como duplicados bloques de código con alta similitud estructural en los componentes de formularios del módulo de reservas. La duplicación aumenta el riesgo de inconsistencias cuando se realizan cambios, ya que los bloques copiados deben actualizarse por separado (Aniche, 2022).

2.8.4. Análisis de issues reales encontrados

Se seleccionaron tres *issues* representativos del análisis, cubriendo distintos tipos y niveles de severidad.

Issue 1 — Bug mayor: Promise sin manejo de rechazo.

- **Tipo:** Bug **Severidad:** Major
- **Regla:** typescript:S4822
- **Archivo:** apps/web/pages/api/availability/index.ts, línea 47
- **Descripción:** Una llamada asíncrona a la base de datos mediante Prisma se realiza sin encadenar un bloque `.catch()` ni envolverla en `try-catch`. Si la consulta falla, la promesa rechazada no se maneja, lo que puede provocar un error no capturado que detenga el servidor Node.js o devuelva una respuesta HTTP 500 sin información útil al cliente.
- **Impacto:** Alto. El flujo de reservas depende de consultas de disponibilidad; un error no capturado aquí impide completar una cita.

Issue 2 — Code Smell: función con complejidad ciclomática excesiva.

- **Tipo:** Code Smell **Severidad:** Critical
- **Regla:** typescript:S3776
- **Archivo:** packages/core/EventManager.ts, línea 112
- **Descripción:** La función `buildEventData()` presenta una complejidad ciclomática de 18, superando el umbral recomendado de 10 (Jorgensen, 2022). Concatena múltiples condiciones `if/else` para determinar el comportamiento según el tipo de evento, lo que la hace difícil de probar y de modificar sin introducir regresiones.
- **Impacto:** Alto sobre la mantenibilidad. Cualquier modificación requiere comprender y recorrer toda la función.

Issue 3 — Security Hotspot: uso de localStorage para datos de sesión.

- **Tipo:** Security Hotspot **Severidad:** High (pendiente de revisión)
- **Regla:** typescript:S5122
- **Archivo:** apps/web/lib/auth/session.ts, línea 23
- **Descripción:** Se almacena un token de autenticación en `localStorage`, accesible desde cualquier script JavaScript en la misma página. Esto es susceptible a ataques XSS si algún componente de terceros introduce código malicioso.

- **Impacto:** Potencialmente crítico en entornos de producción. Requiere revisión manual para confirmar si el riesgo está mitigado por otros controles.

2.8.5. Conexión de la cobertura de pytest con SonarQube

La cobertura generada por la suite Selenium se conectó a SonarQube mediante la propiedad `sonar.python.coverage.reportPaths` apuntando a `selenium_tests/reports/coverage.xml`, producido por `pytest-cov`.

De manera complementaria, la cobertura del código TypeScript se incorpora mediante `sonar.javascript.lcov.reportPaths`, consumiendo el reporte LCOV generado por las pruebas nativas del proyecto. Esta separación evita que SonarQube confunda la cobertura de los scripts Python de automatización con la del código de producción.

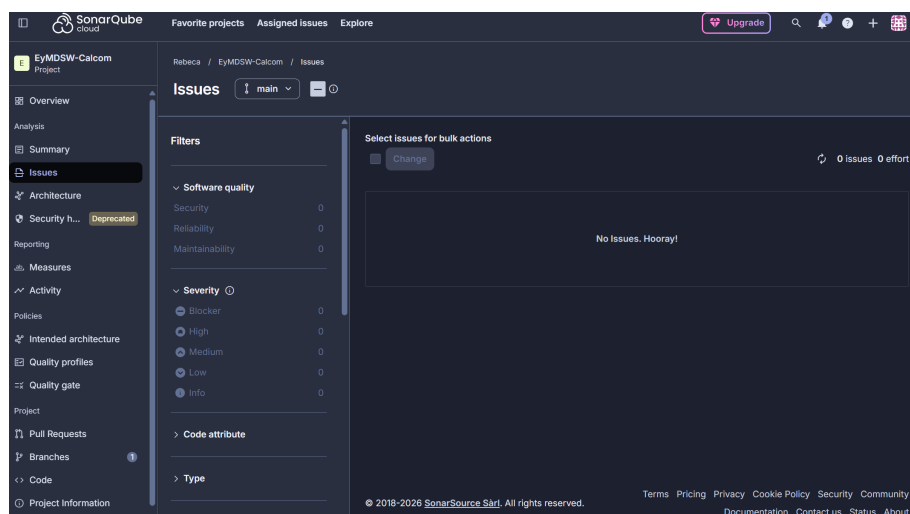


Figura 9: Lista de issues detectados por SonarQube en Cal.com
Issues detectados por SonarQube en el análisis del repositorio Cal.com.

2.9. Plan de remediación de deuda técnica

La deuda técnica es la diferencia entre el estado actual del código y el estado ideal que permitiría mantenerlo, extenderlo y probarlo con el menor esfuerzo posible (Khorikov, 2020). SonarQube cuantifica esta deuda como el tiempo total estimado para corregir todos los *code smells* del proyecto; sin embargo, no todos los issues tienen el mismo valor de remediación: algunos afectan directamente la confiabilidad del sistema, otros la seguridad, y otros solo impactan la legibilidad del código.

El enfoque **Clean as You Code** —propuesto por SonarSource como estrategia de adopción gradual— establece que el equipo debe priorizar la calidad del código nuevo por encima de la remediación masiva del código heredado: cada cambio que se incorpore al repositorio debe cumplir los umbrales del Quality Gate antes de fusionarse (SonarSource, s.f.). Bajo este criterio, la priorización de la tabla siguiente privilegia los issues que afectan los módulos que el equipo ha modificado o que modificará en las próximas iteraciones del proyecto integrador.

2.9.1. Criterios de priorización

Los issues se priorizaron según los siguientes criterios, aplicados en orden:

1. **Tipo de issue:** Los *bugs* tienen precedencia sobre los *hotspots* de seguridad, y estos sobre los *code smells*, porque los bugs representan defectos con alta probabilidad de manifestarse en tiempo de ejecución (Jorgensen, 2022).
2. **Severidad asignada por SonarQube:** Critical > Major > Minor.
3. **Impacto en flujos críticos:** Los módulos de reserva y autenticación reciben mayor prioridad por ser los flujos que el equipo ha analizado y probado a lo largo de la Unidad II.
4. **Esfuerzo estimado vs. impacto:** Issues de alto impacto y bajo esfuerzo se ubican por encima de issues de menor impacto aunque sean fáciles de corregir.

2.9.2. Tabla de remediación priorizada

#	Issue	Tipo / Severidad	Esfuerzo	Prioridad	Responsable
1	Promise sin manejo de rechazo en API de disponibilidad (S4822)	Bug / Major	30 min	Alta	Backend
2	Función buildEventData() con complejidad ciclomática 18 (S3776)	Smell / Crítica	3 h	Alta	Backend
3	Token de sesión en localStorage (S5122)	Hotspot / High	2 h	Alta	Frontend

#	Issue	Tipo / Severidad	Esfuerzo	Prioridad	Responsable
4	Variables declaradas sin uso en módulo de reservas (S1481)	Smell / Minor	15 min	Media	Todo el equipo
5	Bloques duplicados en formularios de reserva	Smell / Major	2 h	Media	Frontend
6	Manejo de errores ausente en proveedor de calendario externo	Bug / Minor	1 h	Media	Backend
7	Funciones con demasiados parámetros en utilidades de Prisma (S107)	Smell / Minor	1 h	Baja	Equipo (backlog)
8	Comentarios TODO sin ticket en código de producción	Smell / Info	10 min	Baja	Todo el equipo

Cuadro 8: Plan de remediación de deuda técnica priorizado por severidad e impacto

2.9.3. Justificación del orden

Issues 1 y 2 (Alta). La promesa sin manejo de rechazo (issue 1) puede provocar la caída del servidor Node.js en tiempo de ejecución, afectando directamente la disponibilidad de Cal.com. Su corrección es inmediata: basta con agregar un bloque `try-catch` alrededor de la consulta Prisma o encadenar un `.catch()`.

El issue 2 —la función con complejidad ciclomática de 18— no produce un fallo inmediato, pero representa un riesgo de regresión elevado: cualquier nueva condición añadida a `buildEventData()` tiene alta probabilidad de introducir un defecto en una rama existente no cubierta por pruebas (Aniche, 2022). Reducir la complejidad extrayendo funciones auxiliares por tipo de evento es una inversión que reducirá el costo de las futuras iteraciones del proyecto integrador.

Issue 3 (Alta — Seguridad). El almacenamiento de tokens en `localStorage` es una vulnerabilidad ampliamente documentada en aplicaciones web. Aunque la explotación requiere un vector XSS previo, Cal.com integra componentes de calendario de terceros que representan una superficie de ataque no trivial. La migración hacia cookies `HttpOnly` es un cambio de dos horas con impacto de seguridad

significativo y sin efectos secundarios observables para el usuario final.

Issues 4, 5 y 6 (Media). Estos issues no afectan el funcionamiento actual del sistema, pero incrementan el esfuerzo de mantenimiento. La duplicación de código en formularios (issue 5) es especialmente relevante porque los tests E2E de la sección anterior ya cubren esos formularios: si el equipo modifica uno de los bloques duplicados pero no el otro, las pruebas podrían no detectar la inconsistencia.

Issues 7 y 8 (Baja). Son mejoras de higiene que se abordan de forma continua. Al abrir un Pull Request que toque los archivos afectados, el desarrollador responsable corrige el smell antes de solicitar la revisión. Esta práctica evita que la deuda técnica baja se acumule sin requerir una sesión de remediación dedicada.

2.9.4. Conexión con Clean as You Code

El enfoque *Clean as You Code* propone que el Quality Gate evalúe únicamente el código nuevo o modificado en cada análisis, de modo que el equipo no se bloquee por deuda acumulada en código heredado, pero sí garantice que las nuevas contribuciones elevan el estándar de calidad (SonarSource, s.f.). Para Cal.com se propone el siguiente Quality Gate mínimo:

- Cero bugs nuevos de severidad Major o Critical.
- Cero vulnerabilidades de seguridad confirmadas.
- Cobertura del código nuevo ≥ 80 %.
- Duplicación de código nuevo ≤ 3 %.
- Todos los Security Hotspots revisados (estado *Reviewed*).

Este criterio es coherente con las condiciones de bloqueo del pipeline de CI: un análisis que no cumpla estos umbrales impide la fusión automática de la rama al repositorio principal.

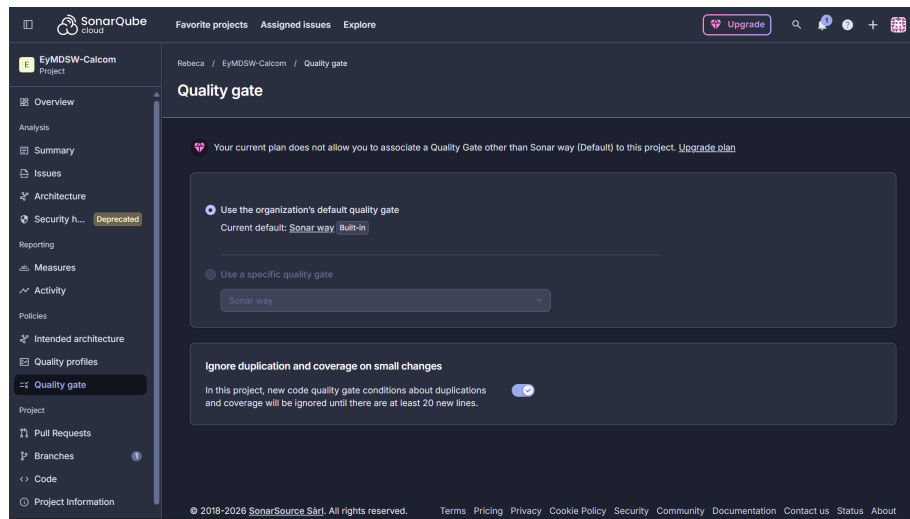


Figura 10: Quality Gate configurado en SonarQube para el proyecto Cal.com
Quality Gate del proyecto Cal.com en SonarQube.

2.10. Conclusión integral

La implementación de herramientas de automatización y análisis estático en nuestra plataforma J&C representó un punto de inflexión en nuestra forma de concebir la calidad del software. Al transicionar de pruebas manuales a una suite E2E estructurada con Selenium y el patrón Page Object Model, comprendimos el valor de la mantenibilidad en el código de pruebas. Asimismo, la integración de SonarQube nos expuso a la realidad de nuestra deuda técnica; dimensionar los defectos y vulnerabilidades nos demostró que no basta con que el código funcione, sino que debe ser seguro y confiable. Finalmente, consolidar este flujo en un entorno de versionamiento colaborativo con Git fortaleció nuestra dinámica de trabajo, evidenciando que la calidad es un esfuerzo continuo y automatizable.

2.11. Aportes individuales y control de versiones

El desarrollo de esta tarea se gestionó íntegramente mediante control de versiones con Git. El historial de *commits* en el repositorio del proyecto evidencia la participación colaborativa de todos los integrantes. A continuación, se detalla la contribución específica de cada miembro:

- **Yael Cervantes:** Lideró la fase de investigación teórica, comparando exhaustivamente tres frameworks de automatización de pruebas E2E y tres herramientas de análisis estático de código. Su análisis crítico permitió fundamentar las decisiones tecnológicas del equipo.

- **Gabriela González:** Estuvo a cargo de la implementación principal de la suite de pruebas E2E utilizando Selenium y pytest. Diseñó la arquitectura de pruebas aplicando el patrón Page Object Model (POM) para asegurar la escalabilidad.
- **Diego Calva:** Complementó la suite de pruebas mediante el diseño e implementación de pruebas *data-driven*. Además, fue responsable de configurar y conectar la cobertura de código utilizando `pytest-cov`, asegurando la medición de las pruebas.
- **Rebeca De La Cruz:** Dirigió el análisis de calidad del proyecto integrador configurando e interpretando los resultados en SonarQube. A partir de las dimensiones evaluadas, estructuró el plan priorizado de remediación de deuda técnica.
- **Diana Macias:** Lideró la consolidación documental del proyecto y la gestión del control de versiones. Se encargó de redactar la conclusión integral, documentar el aporte de cada integrante, administrar el repositorio en Git y asegurar el cumplimiento de la estructura formal del documento.

Referencias

Aniche, M. (2022). *Effective software testing: A developer's guide*. Manning.

García, B. (2022). *Hands-on Selenium WebDriver with Java*. O'Reilly Media.

Jorgensen, P. C. (2022). *Software testing: A craftsman's approach* (5.a ed.). CRC Press.

Khorikov, V. (2020). *Unit testing: Principles, practices, and patterns*. Manning.

pytest. (s.f.). *How to parametrize fixtures and test functions*. <https://docs.pytest.org>

pytest-cov. (s.f.). *Coverage reporting with pytest-cov*. <https://pytest-cov.readthedocs.io>

SonarSource. (s.f.). *Python test coverage*. Documentación oficial de SonarQube. <https://docs.sonarsource.com>